Universitas Esa Unggul
Powered by Arizona State University®

CSF308

OBJECT ORIENTED PROGRAMMING

Sesi 5

Encapsulation with Class (Concept)





Dosen Koordinator:

Dewi Setiowati, A.Md., S.Pd., M.Tr.Kom.

NID:

8413

E-mail:

dewi.setiowati@esaunggul.ac.id

VISI

Menjadi perguruan tinggi kelas dunia berbasis intelektualitas, kreatifitas dan kewirausahaan, yang unggul dalam mutu pengelolaan dan hasil pelaksanaan Tridarma Perguruan Tinggi.

MISI

1. Menyelenggarakan pendidikan tinggi yang bermutu dan relevan
2. Menciptakan suasana akademik yang kondusif
3. Memberikan pelayanan prima kepada seluruh pemangku kepentingan

College Contract

1. Students are required to attend or join the class on time with a maximum **delay of 15 minutes**.
2. Every student's class is required to do a presence **scan QR Code on SIAKAD**
3. Students should attend at least 80% of the scheduled lectures to be able to take the Final test.

Assessment

Coursework evaluation will be weighted as follows:

➢ Mid-Term Examination	10%
➢ Final Examination	10 %
➢ Final Project	50%
➢ Others (class participation 10%, quiz 10%, practical assignments 10%)	30 %
Total	100%

Topik Sebelum UTS

Sesi 1

Introduction to Object Oriented Programming

Sesi 2

Data Type, Variable, Method and Expression

Sesi 3

Control Statement

Sesi 4

Function and Parameter

Sesi 5

Encapsulation (Concept)

Sesi 6

Implementation Encapsulations In Case Study

Sesi 7

Pointer and Memory Management

Ujian Tengah Semester

Topik Sebelum UAS

Sesi 8

Inheritance Basic

Sesi 9

Inheritance Details

(Case Study)

Sesi 10

Polymorphism

Sesi 11

Exception Handling

Sesi 12

Standard Template Library

Sesi 13

Class Presentation

Sesi 14

Class Presentation and Cource Review

Ujian Akhir Semester

Capaian Pembelajaran

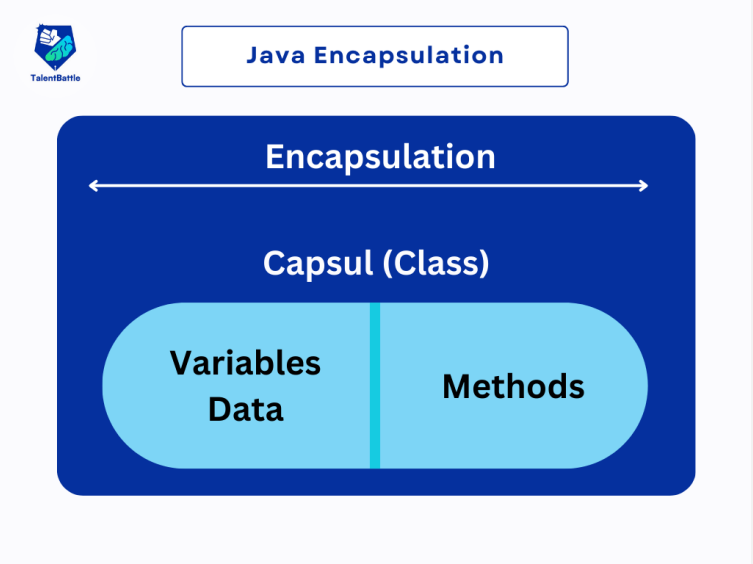
Setelah mempelajari materi pada pertemuan ini, diharapkan mahasiswa :

1. Mampu memahami *Encapsulation with Class (Concept)*
2. Mampu membuat dan mengimplementasikan *Encapsulation with Class (Concept)* ke dalam sebuah aplikasi dalam Pemrograman Java (NetBeans)

Pendahuluan

Enkapsulasi adalah proses pembungkusan (encapsulation) dari suatu kelas atau biasa disebut ***information hiding***.

Membuat terlindungi atau ada batasan di **class** yg telah kita buat.





ENCAPSULATION

CLASS ?

Kelas adalah *blueprint* yang mendefinisikan variabel dan methods dari suatu kategori.

Variabel -> Status
Methods -> Perilaku.

Class adalah konsep fundamental lain dalam bahasa berorientasi objek yang menyediakan *Blueprint* untuk tipe objek baru ('klasifikasi').

- Sebuah kelas menguraikan data, fungsi dan objek antarmuka yang akan diterima kelas itu.
- Sebuah kelas juga mendefinisikan bagaimana objek kelas tersebut berperilaku dengan menyediakan kode yang mengimplementasikan fungsi-fungsi yang terkait dengan kelas tersebut.
- Seorang programmer dapat membuat satu atau lebih objek dari suatu kelas

OBJECTS?

- Objek adalah turunan dari kelas tertentu
- Objek adalah kumpulan yang berisi **variabel** dan **methods** terkait.

ABSTRACTION

- abstraksi: Jarak antara ide dan detail.

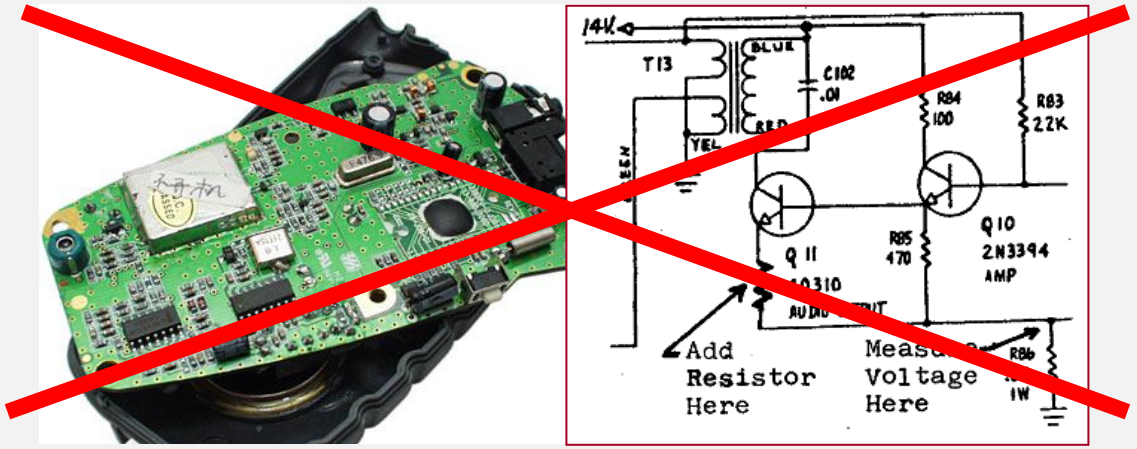
  Kita dapat menggunakan objek tanpa mengetahui cara kerjanya.

- abstraksi di iPod:

  Anda memahami perilaku eksternalnya (tombol, layar).

  Anda tidak memahami detail dalamnya, dan Anda tidak perlu memahaminya.



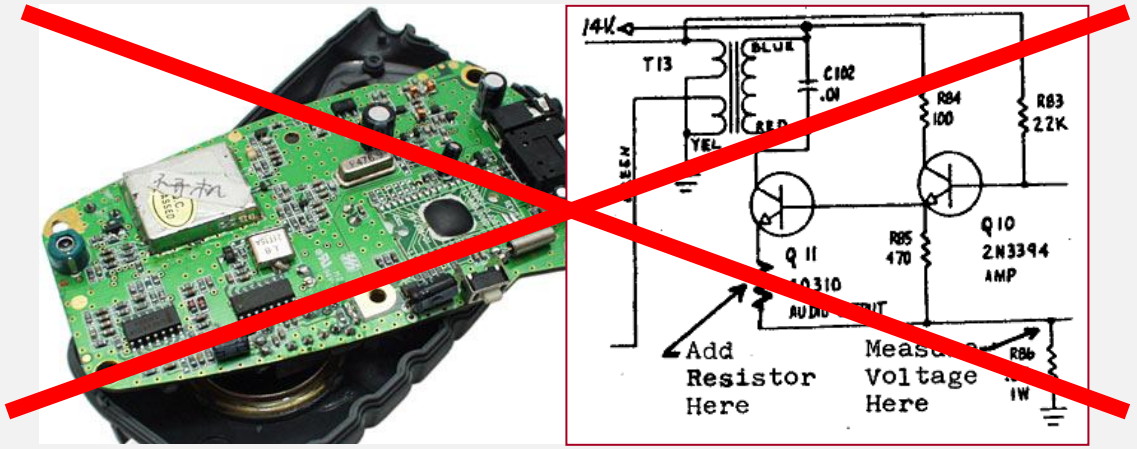


ENCAPSULATION

Menyembunyikan detail implementasi dari klien/user.

Class menggunakan konsep abstraksi
Enkapsulasi memaksa abstraksi.
memisahkan pandangan eksternal (perilaku) dari pandangan internal (keadaan)
melindungi integritas data objek



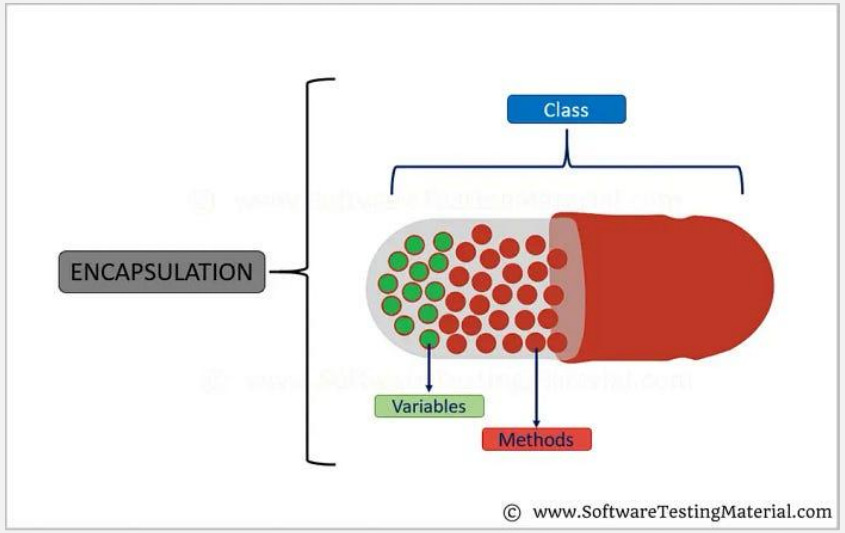


ENCAPSULATION

Pembungkusan variabel (data) dan methods menjadi satu kesatuan yang disebut **class** merupakan **enkapsulasi**.

Untuk mencapai ini, Anda harus :

- **Mendeklarasikan atribut/variabel sebagai private**
- **Menyediakan method publik get dan set untuk mengakses dan memperbarui nilai dari variabel private**



ENCAPSULATION

Terdapat tiga tingkat akses yang terkait dengan enkapsulasi, yaitu:

- **Private**

  Ketika mendeklarasikan data dan method dengan private, maka data dan method tersebut hanya dapat diakses oleh kelas yang memilikinya saja.

- **Protected**

  Ketika mendeklarasikan data dan method dengan protected, maka data dan method tersebut dapat diakses oleh kelas yang memilikinya dan kelas-kelas yang masih memiliki hubungan turunan.

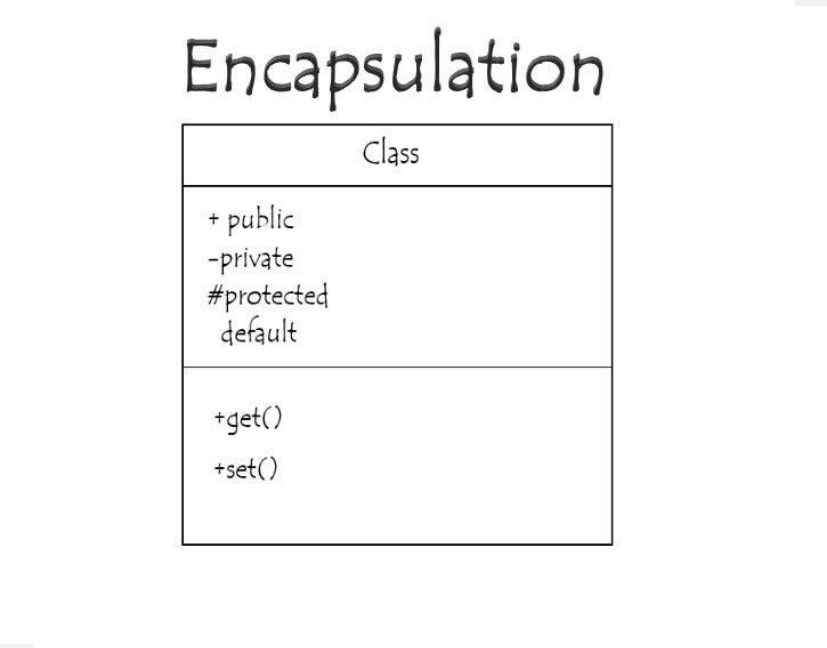
- **Public**

  Data dan method yang bersifat public akan dapat diakses oleh semua bagian dalam program. Semua bagian dalam program adalah semua kelas yang memiliki hubungan turunan maupun yang tidak memiliki hubungan sama sekali

ENCAPSULATION

Perbedaan Tingkat Akses Enkapsulasi

	Private	Protected	Public
Akses dalam satu kelas	Bisa	Bisa	Bisa
Akses dalam kelas turunan	Tidak	Bisa	Bisa
Akses dalam kelas bukan turunan	Tidak	Tidak	Bisa



GETTER AND SETTER METHODS

Saat mendefinisikan suatu objek (atau, me-encapsulate variabel suatu class), biasanya dideklarasikan semua variabel sebagai **private**.

Setter methods adalah methods untuk **mengatur/memodifikasiupdate (write only)** nilai variabel dari private suatu class.

Getter methods adalah methods untuk **mengakses/mengambil (read olnly)** nilai variabel dari **private** dikenal sebagai methods **pengambil**.

BENEFIT OF ENCAPSULATION

- Kontrol yang lebih baik untuk atribut/variabel dan method class
- Atribut class dapat dibuat **read-only** (jika Anda hanya menggunakan method get), atau **write-only** (jika Anda hanya menggunakan method set)
- Fleksibel karena programmer dapat mengubah satu bagian dari kode tanpa mempengaruhi bagian lainnya
- Meningkatkan keamanan data

CONSTRUCTOR

Konstruktor adalah fungsi anggota khusus dalam suatu kelas yang tujuannya biasanya untuk menginisialisasi anggota suatu objek.

Konstruktor mudah dikenali karena:

- Namanya sama dengan kelasnya.
- Ia tidak memiliki tipe pengembalian.

ENCAPSULATION

Contoh program :

Pertemuan51.java | main.java

Source | History

```
*/
public class Pertemuan51 {

    /**
     * @param args the command line arguments
     */
    //mendeklarasikan data dan method dengan private
    private String nama;
    private int nim;

    //getter
    public String getNama() {
        return this.nama;
    }

    //setter
    public void setNama(String nama) {
        this.nama = nama;
    }
}
```

Pertemuan51.java | main.java

Source | History

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/License
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes
 */
package pertemuan5.pkg1;

/**
 *
 * @author TOSHIBA
 */
public class main {
    public static void main(String[] args) {
        // TODO code application logic here
        Pertemuan51 objek = new Pertemuan51();
        objek.setNama("John Smith");
        System.out.println("Nama : " + objek.getNama());

    }
}
```

Output

Pertemuan5.1 (debug) | Debugger Console

```
debug:
Nama : John Smith
BUILD SUCCESSFUL (total time: 0 seconds)
```

ENCAPSULATION

Class Luas Segitiga :

```
 9      * @author TOSHIBA
10      */
11     public class LuasSegitiga {
12         // atribut
13         private int alas;
14         private int tinggi;
15         private double luas;
16
17         // setter method untuk alas
18         public void setAlas(int a){
19             if (a > 0){
20                 this.alas = a;
21             } else {
22                 this.alas = 0;
23             }
24         }
25
26         // setter method untuk tinggi
27         public void setTinggi(int t){
28             if (t > 0){
29                 this.tinggi = t;
30             } else {
31                 this.tinggi = 0;
32             }
33         }
```

```
34
35         // getter method untuk luas
36         public double getLuas(){
37             // hitung luasnya
38             this.luas = this.alas * this.tinggi * 0.5;
39             return this.luas;
40         }
41     }
42
```

ENCAPSULATION

Class Persegi Panjang :

```
Source  History
 *
 * @author TOSHIBA
 */
public class PersegiPanjang {
    // atribut
    private int panjang;
    private int lebar;
    private double luas;

    // setter method untuk panjang
    public void setPanjang(int p){
        if (p > 0){
            this.panjang = p;
        } else {
            this.panjang = 0;
        }
    }

    // setter method untuk lebar
    public void setLebar(int l){
        if (l > 0){
            this.lebar = l;
        } else {
            this.lebar = 0;
        }
    }
```

```
    // setter method untuk lebar
    public void setLebar(int l){
        if (l > 0){
            this.lebar = l;
        } else {
            this.lebar = 0;
        }
    }

    // getter method untuk luas
    public double getLuas(){
        // hitung luasnya
        this.luas = this.panjang * this.lebar;
        return this.luas;
    }
}
```

ENCAPSULATION

Class Luas Lingkaran:

Source History

```
/**
 *
 * @author TOSHIBA
 */
public class LuasLingkaran {
  // atribut
    private int jejari;
    private double luas;

    // setter method untuk jejari
    public void setJejari(int r){
        if (r > 0){
            this.jejari = r;
        } else {
            this.jejari = 0;
        }
    }

    // getter method untuk luas
    public double getLuas(){
        // hitung luasnya
        this.luas = Math.PI * Math.pow(this.jejari, 2);
        return this.luas;
    }
}
```

ENCAPSULATION

Class Main :

```
 * @author TOSHIBA
 */
public class Main {
    public static void main(String[] args) {
        // hitung luas segitiga
        LuasSegitiga stl = new LuasSegitiga();
        stl.setAlas(2);
        stl.setTinggi(3);
        double luassegitiga = stl.getLuas();

        // hitung luas persegipanjang
        PersegiPanjang ppl = new PersegiPanjang();
        ppl.setPanjang(4);
        ppl.setLebar(2);
        double luaspersegipanjang = ppl.getLuas();

        // hitung luas setengah lingkaran
        LuasLingkaran ling = new LuasLingkaran();
        ling.setJejari(2);
        double luaslingkaran = 0.5 * ling.getLuas();

        // hitung luas daerah diarsir
        double luasArsir = luassegitiga  + luaspersegipanjang - luaslingkaran;
        System.out.println("Luas daerah diarsir: " + luasArsir + " cm2");
    }
}
```

```
Output ×
Pertemuan52_luassegitiga (debug) ×   Debugger Console ×
debug:
Luas daerah diarsir: 4.716814692820414 cm2
BUILD SUCCESSFUL (total time: 0 seconds)
```

TASK

Implementasikan **encapsulation** dalam program yang telah Anda buat masing”

Universitas Esa Unggul
Powered by Arizona State University®

Terima Kasih

